

Prog 4 – pótpótZH

Bevezetés

A bevezetést és a feladatokat figyelmesen olvassa végig.

A ZH-t zip fájlként kell feltölteni az alábbi formátumban: `név_neptun_kód.zip` (tartalmazza a potzh-feladat és az elkészített potzh-ws-feladat projektet is)

Az alkalmazáserver helye a labor gépein: `C:\apache-tomcat`

Nem forduló kód (maven build használatával) vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott kód alapján) szintén 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér.

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git hiánya esetén a ZH 0 pontos.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során (figyeljen arra, hogy egy feladatot egyben commitálja).

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így a lentebb megadott Resource segítségével működő adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

Ha a docker container létrehozása esetén hibaüzenetet kapna, akkor a docker desktop-on jobb egérgombot nyomva válassza a „Switch to linux containers” lehetőséget, majd a docker beállításai között válassza a „Docker Engine” menüpontot és konfigurációként állítsa be a következőt:

```
{  
  "builder": {  
    "gc": {  
      "defaultKeepStorage": "20GB",  
      "enabled": true  
    }  
  },  
  "experimental": true  
}
```

Ellenőrizze, hogy a mariadb-java-client-3.1.3.jar megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsse le az internetről és helyezze el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítja a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolathoz szükséges resource definíciója (ha már van másik resource felvéve azt törölje ki):

```
<Resource name="jdbc/pot-potZH-MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/potpotzh_database" username="potpotzh_user"
password="PotPotZHPassword111" maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban található felhasználó és szerepkör is.

A felhasználó jelszava "Password111". A jelszó Bcrypttel van elforgatva.

Git (potzh-feladat és potzh-ws-feladat)

- Hozzon létre egy git repository-t a projekteken.
- Készítsen egy develop branchet, majd ezen a branchen dolgozzon.
- Minden feladat megoldását külön-külön commitálja el, figyeljen a beszédes commit message-kre.

A beadandónak három alkalmazást kell tartalmaznia:

- potpotzh-feladat
- potpotzh-ws-feladat
- potpotzh-client-feladat

Az adatbázist használva készítse el az alkalmazásokat.

Készítsen egy alkalmazást **potpotzh-ws-feladat** néven, ami futtatható legyen a tomcat alkalmazásszerveren.

A web service kérésben várjon egy cég azonosítót és adja vissza válaszban, hogy mennyi eladott terméke van az adott cégnek.

Az érték generálásához használja az alábbi kódrészletet:

```
private final Map<String, Double> map = new HashMap<>();
```

```
public CompanyResponse getCompanyData(CompanyRequest request) {  
    return new CompanyResponse(request.getCompanyId(), this.map.computeIfAbsent(request.getCompanyId(),  
companyId -> ThreadLocalRandom.current().nextLong(100000000L, 100000000000L)));  
}
```

A web service a /ws/company-data url-pattern-en legyen elérhető.

Az elkészített web service-ből generáljon egy WSDL leíró az /src/main/resources/wsdl mappába.

Készítse el a **potpotzh-feladat** alkalmazást, ami szintén tomcaten fusson.

A korábban elkészült WSDL-t használva a potpotzh-feladat projektben generáljon klienst a web service-hez.

Készítse el a cég repository-t, ahol lehessen a cégeket listázni, egyesével lekérdezni és újat menteni.

Készítse el a fenti repository-hoz a service rétegbeli metódusokat.

Készítsen egy service metódust, ami a generált web service klienst használva meghívja a web service-t.

Készítse el a service-hez a Controller rétegbeli metódusokat, ez kommunikáljon JSON-kkel és mindig adjon vissza választ.

Készítsen egy cég listázó oldalt, ahol látható a cég id-ja, neve, alapítási éve, ismert termékének neve, illetve legyen egy XML gomb, amire ha kattintunk, akkor a lista alatt tag library-ban megjelennek xml formátumban az adott cégnek az adatai.

Készítse el az autentikációt a potpotzh-feladat alkalmazásra:

FORM alapú autentikáció legyen.

A REST apik legyenek elérhetőek bejelentkezés nélkül.

A listázó oldal csak „viewer” szerepkörrel legyen elérhető.

Legyen hibás bejelentkezésnek saját oldala.

Készítse el a **potpotzh-client-feladat** projektet, ami maven alapú, de nem web alkalmazás.

Az alkalmazás minden indításkor hívja meg külön-külön teszt adatokkal a potpotzh-feladat Controller-ben elkészített listázó, id alapján lekérdező és eladott termékek számát visszaadó REST API endpointokat és írja ki az eredményt output-ra.